

Philipps University of Marburg
Department of Mathematics and Computer Science

Bachelor's Thesis for B.Sc. Data Science

Empirical Evaluation of Tree Extraction

Julius Reitlinger (3766624)

2026-08-17

First Supervisor: Prof. Dr. Sebastian Wild
Second Supervisor: Prof. Dr. Bernhard Seeger

Abstract: Often topological graph analysis holds more value than specific vertex labels; therefore, recent research has laid focus on lossless compression of unlabeled graphs. TREX (Tree Extraction) is a novel succinct representation that achieves exactly this by partitioning a graph into a spanning tree and the residual graph, introducing canonical relabelings of the vertices [ANNW26]. This thesis provides a comprehensive empirical evaluation of TREX.

To achieve this, a high-level implementation was built to serve as a framework for future succinct implementation, verify the navigational logic of TREX queries – revealing and resolving some flaws of the directed queries in the process – and extend the logic to undirected graphs, which was previously unspecified. On top of this structure, an evaluation pipeline was built to benchmark TREX and its planar subgraph extension PLEX (Planar Subgraph Extraction) – whose idea is simply to extract more edges, which can be stored in a low bits-per-edge environment. The evaluation showed that TREX consistently reduces space compared to the wavelet tree representation of a graph. PLEX, on the other hand, was able to further improve the results, but at the moment fell short of expectations due to the lack of a good planar subgraph extraction algorithm. Furthermore, different greedy approaches for MINE-TREX and U-MINETREX (Undirected Minimum-Entropy Tree-Extraction) were evaluated. While they delivered the best results throughout, we saw that the primary driver of graph compression stems from the edge extraction itself, rather than algorithms minimizing entropy. Building upon this knowledge, an $\mathcal{O}(1)$ density-based predictor of TREX compression potential was developed, generalizing well on unseen data despite its simplicity, but lacking reliability in edge cases.

Contents

1. Introduction	4
1.1. Contributions	4
2. Preliminaries	5
2.1. Bitvectors	5
2.2. LOUDS	6
2.3. Wavelet Trees	7
2.4. Tree Extraction	7
2.5. Theoretical Bounds	9
2.6. Densest Subgraph Problem (DSP)	11
2.7. Maximum Planar Subgraph Problem (MPS)	12
3. Implementation	14
3.1. Architecture and Graph Construction	14
3.2. Navigational Logic and Undirected Generalization	16
4. Empirical Evaluation	19
4.1. Methodology	19
4.2. Datasets	20
4.3. Results and Discussion	22
4.3.1. Base Compression Performance	22
4.3.2. Impact of Edge Orientation	23
4.3.3. Greedy MINETREX approximation	25
4.3.4. Compression-Potential Heuristic	25
4.3.5. Theoretical Bounds	27
4.3.6. Planar Extraction	29
4.3.7. Twin Removal	29
5. Conclusion	30
5.1. Summary	30
5.2. Future Work	31
A. Directed Queries	36
B. Undirected Queries	42

1. Introduction

In recent years, networks have been constantly growing in size and so has the need to develop succinct data structures around them. For many practical network analysis applications, the structural topology is of greater interest, while the specific original labels of the vertices are rather secondary. That is why recent research has laid focus on lossless compression of unlabeled graphs. TREX (Tree Extraction) is such a succinct representation and has been recently introduced by Alaoui et al. [ANNW26]. The concept relies on partitioning a graph into two components: a spanning tree (or forest) and the remaining edges. The extracted forest can be stored very succinctly, and equally importantly, induces a deterministic, canonical relabeling of the vertices. The residual graph is then stored separately using an adjacency string, compressed to its zeroth-order empirical entropy. Because the entropy of the adjacency string is strictly tied to the distribution of its letters, the specific choice of the removed forest plays an important role in the resulting space consumption.

The problem of finding the optimal tree, introduced as (undirected) Minimum-Entropy Tree-Extraction ((U-)MINETREX) [ANNW26], is NP-hard. However, a simple greedy approach has been proven to offer an optimal additive linear term for the undirected and directed problems.¹ While the theoretical foundations of TREX and in particular MINETREX have been established, their practical impact on real-world networks is yet to be explored and requires comprehensive empirical evaluation. This thesis aims to close exactly this gap.

1.1. Contributions

To test whether all different concepts work together as a cohesive unit, a high-level proof-of-concept implementation of the TREX data structure was built with extended support for undirected graph queries. In the process, some minor errors in the original directed query logic were found and corrected. It is implemented in Python and based on dummy implementations for the three core underlying structures. Because the components only interact through common interfaces, they can be seamlessly replaced with fully succinct implementations to make it a productive implementation. The resulting structure is then analyzed on a diverse set of benchmark datasets, demonstrating consistent space reductions over standard wavelet tree baselines [Nav14]. The evaluation also quantified the tightness of the existing bounds for the residual graph’s absolute empirical entropy introduced by Alaoui et al. [ANNW26], showing that they mostly overestimate the real entropy by $n \log_2 n$ bits. Furthermore, to isolate the origin of TREX compression and the impact of different approximation strategies, the greedy edge orientation for undirected graphs as well as the greedy MINETREX approximation are benchmarked against different baselines. The results revealed that the greedy approaches performed best throughout, but also that they are not the primary driver of compression, which instead

¹For U-MINETREX, only a preprocessing step of choosing one directed edge to store for each undirected edge is required.

is the raw edge extraction itself. Following from those insights, PLEX (Planar Subgraph Extraction) was evaluated — an extension of TREX that combines a linear-time planar subgraph approximation [CFFK98] with a succinct block-cactus representation [CJSS24] to extract additional edges. It showed consistent improvements over TREX’s compression performance on most datasets. Additionally, a new empirically validated density-based heuristic was developed. It accurately estimates the TREX compression potential of a given graph solely based on its density ahead of TREX construction.

Finally, the potential of twin removal (merging vertices with identical neighborhoods) was explored and was shown to be highly domain-specific.

Outline: The remainder of this thesis is structured as follows. Section 2 establishes the preliminary concepts, explaining the utilized underlying structures and laying the conceptual foundation of TREX and its planar subgraph extraction. Section 3 describes the structure of the Python implementation and highlights corrected query logic. Section 4 includes the complete empirical evaluation, presenting the methodology and discussing the various results. Finally, Section 5 summarizes all findings of this thesis and guides potential directions for future research.

2. Preliminaries

2.1. Bitvectors

A bitvector $B[0, n - 1]$ is a sequence of bits that can be stored compactly and queried in constant time. They are the most fundamental building blocks for succinct data structures and are used in a variety of applications. The three main operations we are interested in are:

- **access**(B, i): Returns the bit at position i in the bitvector $B[0, n - 1]$.
- **rank** $_v(B, i)$: Returns the number of times the bit $v \in \{0, 1\}$ appears in the prefix $B[0, i]$. For $v = 1$, it works essentially like a partial sum if we think of bits as numbers. By convention **rank** $_v(B, i) = 0$ for $i < 0$.
- **select** $_v(B, j)$: The inverse of **rank**. It returns the position of the j -th occurrence of a bit $v \in \{0, 1\}$ in B . For example, if $j = 1$, it gives us the position of the first v .

All of those operations can be implemented in constant time while only costing $o(n)$ bits of additional space and simultaneously storing the bitvector itself within $n\mathcal{H}_0(B) + o(n)$ bits using the (c, o) structure introduced with RRR [RRS07], where, for a sequence $S \in \Sigma^n$, $\mathcal{H}_0(S)$ denotes its zeroth order empirical entropy:

$$\mathcal{H}_0(S) = \sum_{c \in \Sigma} \frac{n_c}{n} \log_2 \left(\frac{n}{n_c} \right)$$

with n_c as the number of occurrences of a letter c .

RRR compression, at its core, simply works by dividing the bitvector into smaller blocks and remembering what a block looks like using only two small identifiers, leading to efficient compression, if B has a skewed distribution of bits. For further details on bitvectors or any other succinct data structure used in this work, see Chapter 4 of Navarro's "Compact Data Structures" [Nav16].

2.2. LOUDS

Because the core idea of TREX is to extract a spanning tree, we need an efficient representation for one. That is exactly where LOUDS (Level-Order Unary Degree Sequence) [Jac89] come in, they require only two bits per edge. To achieve this, they store one long bitvector. This bitvector is constructed by first adding 10 (for edge cases) to the beginning, then (in BFS order) for each node, add c ones followed by a single zero, where c is the number of children a node has. This results in a single bitvector representing the complete topology of the tree, enabling constant-time queries such as asking for the degree, parent, or child of a node. Those are also already the three essential operations needed for TREX navigation,² many more can be easily supported by LOUDS as well. A short example of a constant-time query to the i -th child of a node j is shown in Figure 1.

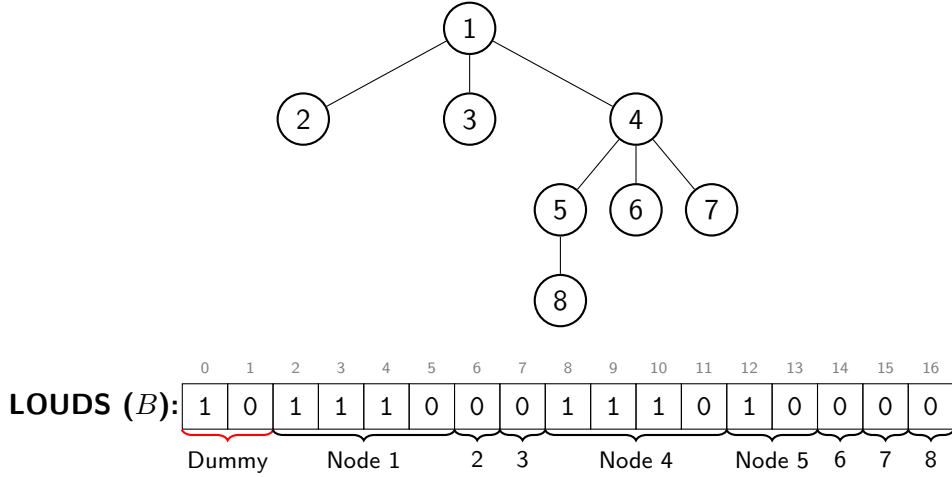


Figure 1: Exemplary constant time LOUDS query: $\text{child}(4,3)$. To determine the third child of node four, we first have to find the absolute position where the listing of node 4's children begins. This is done by querying $\text{select}_0(B, 4) = 7$. From this, we know that the third child must lie at position $7 + 3 = 10$. Finally, to determine what node is represented at position 10, we simply calculate the number of ones before or at position 10, giving us the final result: $\text{rank}_1(B, 10) = 7$.

The $2n + o(n)$ space requirement is quite intuitively achieved as well; B has $\sim n$ ones because each node (except the root) is exactly one other node's child (being a one in their

²For undirected TREX, **child-rank**, the inverse of **child** is an additional requirement, but can also be easily supported.

"children-list"). Also then B contains n zeros, coming from each node, even a leaf, having a "children-list", which always ends with a zero. On top, only the $o(n)$ bits of overhead are required to support constant time queries on the bitvector. If the tree has a skewed degree distribution, the bitvector can again be further compressed to its zeroth-order empirical entropy.

2.3. Wavelet Trees

Our chosen base representation of a graph consists of combining an adjacency string with a bitvector, as we will discuss later on. Therefore, a very crucial structure for the TREX representation is a wavelet tree [GGV02]. It enables storing such a sequence of integers/characters succinctly, while also allowing for efficient queries. Those needed queries mirror the behavior of a bitvector, only now across a larger alphabet:

For an integer alphabet Σ of size σ and a sequence $A[0, \dots, n-1] \in \Sigma^n$, a wavelet tree supports the $\mathcal{O}(\log_2 \sigma)$ operations **access**, **rank**³ and **select**, enabling us to for example ask where the i -th occurrence of a letter $v \in \Sigma$ appears, while taking only $n\mathcal{H}_0(A) + o(n)$ bits [Nav14].

The construction of a wavelet tree utilizes a recursive partitioning of the sequence $A[0, \dots, n-1] \subseteq \Sigma^n$:

- A bitvector $B[0, m-1]$ is created as a root node. Based on its natural ordering, each character $v \in \Sigma$ is assigned a 0 if and only if $v \leq \lfloor \frac{a+b}{2} \rfloor$, where a and b are the minimum and maximum integer values in the sequence.
- Each character assigned a 0 now goes to the left child node, and those assigned a 1 to the right child node.
- This binary split is done recursively for each child, until all leaves would consist of uniform bitvectors and correspond to exactly one symbol in Σ .

Figure 2 illustrates this construction on a short example sequence, while also explaining how a single **select** query is executed.

2.4. Tree Extraction

Tree Extraction (TREX) is a recently introduced lossless graph compression scheme [ANNW26]. It utilizes two main principles: the assignment of canonical vertex labels solely given by the graph structure and the entropy-based compression of adjacency lists.

Let $G = (V, E)$ be a directed, connected graph with $n = |V|$ vertices and $m = |E|$ edges. The core concept of TREX is to partition such a graph into an undirected spanning tree T as well as the residual graph $G - T$. By extracting T , $n - 1$ edges can be represented highly succinctly, only costing $\sim 3n$ bits. Those $3n$ bits come from two different sources: $2n$ bits from the undirected LOUDS tree's topology, as seen in Section 2.2, and another n bits from the direction bitvector D , which records the original orientation of each edge,

³As for bitvectors, we use the convention $\mathbf{rank}_v(A, i) = 0$ for $i < 0$.

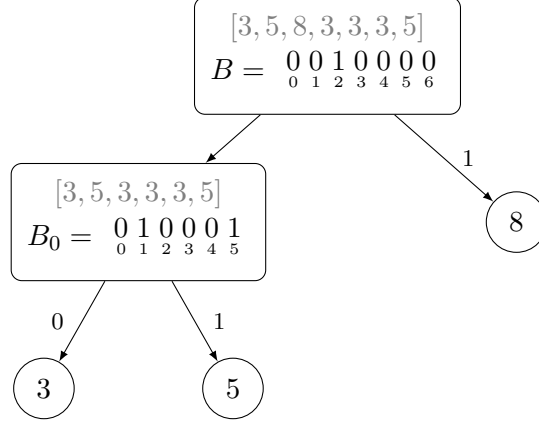


Figure 2: Exemplary wavelet tree for the string $A = [3, 5, 8, 3, 3, 3, 5]$. While of course not being stored explicitly, the conceptual string for each bitvector is shown in grey. To evaluate the query $\text{select}_8(A, 1)$, we first conclude that $8 > \lfloor \frac{8+3}{2} \rfloor$, therefore it is assigned a 1. Now we recurse with the second child, where we in this case already arrive at a leaf node, correctly giving us the final result at $\text{select}_1(B, 1) = 2$. This tree also ideally proves how the skewed character distribution in the string leads to skewed bitvectors, allowing for efficient compression.

where $D[v-1] = 0$ if and only if the edge between v and v 's parent points upwards.⁴ This is significantly cheaper than storing the edges via a more universal graph representation.

Additionally, a major advantage is that the extracted spanning tree T can be used to deduce a canonical, deterministic relabeling of the vertices. This is done via the indices induced by a level-order traversal of the tree T . If we were to restrict ourselves to labeled graphs, we would have to memorize a vertex label mapping taking $\log_2(n!) \approx n \log_2(n)$ bits, effectively defeating the purpose of the compression [ANNW26].

Now that a tree is extracted and the vertices are renamed, we must efficiently store the residual graph $G - T$. We represent the residual graph, or a directed graph G in general, using an adjacency string $A[0, m-1]$ and a complementary bitvector $S[0, n+m-1]$. For each vertex, we add all of its j outneighbors to A and a 1 followed by exactly j zeros to S . The purpose of S is to identify where each node's outneighbor list starts within A .⁵ And since S is a bitvector of length $n+m$ containing n ones, its compressed space requirement is $\log_2 \binom{n+m}{n} + o(n+m)$ for efficient query support. A itself, represented as a fixed-size array, uses $m \lceil \log_2 n \rceil$ bits, but by switching this out with a compressed wavelet tree we can get this down to $m \mathcal{H}_0(A)$ [Nav14]. This space reduction also introduces a query-time trade-off, since wavelet tree navigation leads to a time complexity of $\mathcal{O}(\log n)$ for most TREX operations, compared to the $\mathcal{O}(1)$ a plain array-based adjacency list

⁴Only the first bit is always zero, due to the root having no parent. This placeholder could also be omitted to save one additional bit.

⁵The developed implementation includes a trailing 1 in S , implementing the convention of $\text{select}_1(S, n+1) = n+m$ and resolving some edge cases. This could be omitted in a succinct implementation, by utilizing that m and n are known by surrounding structures.

would achieve.⁶

Because A contains exactly the outneighbors of our graph, the symbol frequencies correspond to the indegrees of the different vertices. Therefore $\mathcal{H}_0(A)$ is identical to the zeroth-order empirical indegree entropy of the corresponding graph G , formally defined as:

$$\mathcal{H}_{deg}^{in}(G) := \sum_{v \in V} \frac{\text{indeg}(v)}{m} \log_2 \left(\frac{m}{\text{indeg}(v)} \right)$$

Accordingly, the absolute indegree entropy of a graph G is denoted as $H(G) = m\mathcal{H}_{deg}^{in}(G)$

This immediately tells us that the space needed for A strictly correlates with the indegree distribution of our graph. Therefore, the specific choice of the removed tree T plays an important role in the resulting space of the residual graph $G - T$.

This brings us directly to the problem of MINETREX (Minimum-Entropy Tree-Extraction), whose goal is to extract an optimal tree T , such that the remaining graph has minimal indegree entropy $H(G - T)$ among all possible spanning trees.

Finding such spanning tree has been proven to be NP-hard [ANNW26]. However, Alaoui et al. introduced a greedy MINETREX approximation that achieves an optimal additive error of at most $(n - 1)/\ln 2$ bits over the optimal solution [ANNW26].

Because the approximated independent cost one saves when removing an edge (u, v) is $\log_2(m/\text{indeg}(v))$ and due to the monotonicity of the logarithm, the greedy approach assigns each edge a weight of $\text{indeg}(v)$. Because lower weights generally represent higher savings, a simple minimum spanning tree algorithm can be performed to achieve this reliable approximation.

This concept naturally extends to undirected graphs as well (U-MINETREX). Since wavelet trees allow for bidirectional navigation, it is sufficient to store one direction for each undirected edge. This gives us additional potential for optimization by orienting the edges in a way that skews the indegree distribution. Therefore, the core approach of U-MINETREX to minimize entropy is to first orient every edge $\{u, v\} \in G$ towards the endpoint with the higher degree, increasing the indegree of already high-degree vertices [CFJ08]. A standard minimum spanning tree is then extracted from this resulting directed graph G' . This greedy pipeline was also shown to achieve an additive error of at most $(m + n - 1)/\ln 2$ bits.

2.5. Theoretical Bounds

Alaoui et al. established two theoretical guarantees for space savings achieved by extracting a spanning tree determined by their simple greedy approximation [ANNW26]. These bounds are essential to provide a mathematically based estimate that can, for example, be used for choosing a specific graph representation upon compression. We introduce those guarantees as the Density Uniformity Bound (DUB) and the Non-Zero Indegree Bound (NIB). Both of them rely on a parameter α , which, for the DUB is determined by the density uniformity of the graph. Let $\delta(G) := \frac{m}{n}$ be the density of

⁶Only $\text{outdegree}(v)$ is $\mathcal{O}(1)$, because no navigation of A' is required.

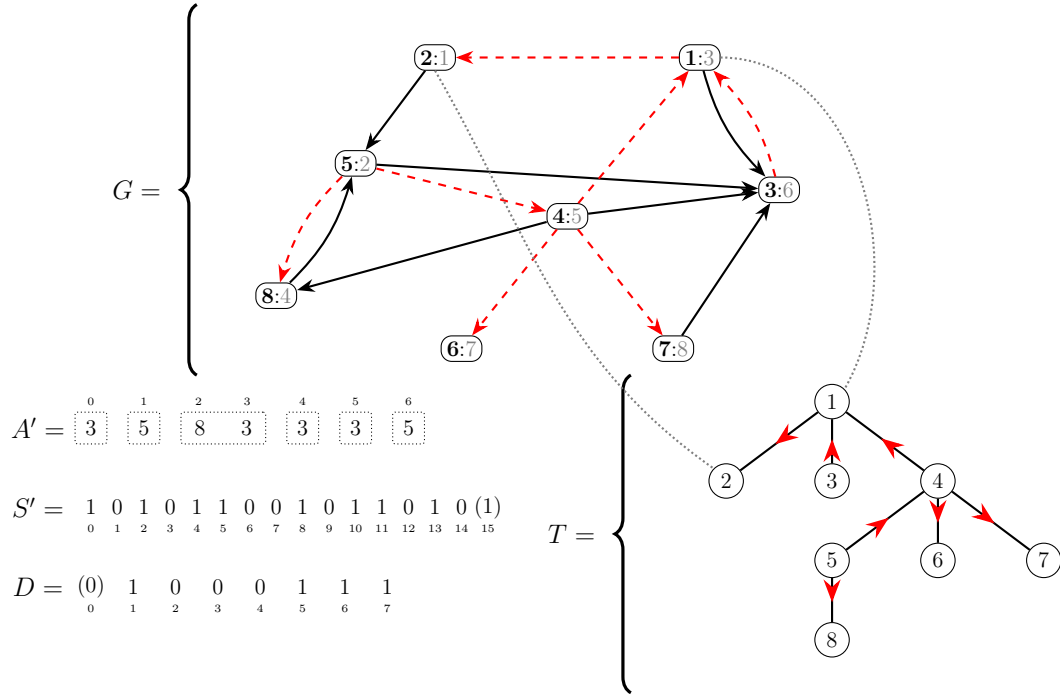


Figure 3: TREX representation of a directed graph G , adapted from Alaoui et al. [ANNW26]. The graph G is represented up top, where dashed red edges make the greedily removed MST (arbitrarily rooted and shown on the right), grey names mark the original nodes labelings. A' as the adjacency string is navigated and stored as seen in section 2.3.

a given graph $G(V, E)$,⁷ α_{DUB} is then defined such that $\delta(G') \leq \alpha_{\text{DUB}}\delta(G)$ for any subgraph $G' \subseteq G$. Given this, Alaoui et al. showed that there exists a spanning tree T , satisfying the following equation (Theorem 1.6 in [ANNW26]):

$$H(G - T) \leq H(G) - \frac{n}{2\alpha_{\text{DUB}}} \mathcal{H}_{\text{deg}}^{\text{in}}(G) + \frac{2n}{\ln 2} \quad (\text{DUB})$$

The NIB follows the exact same formula, but replaces the density-based uniformity parameter α_{DUB} with α_{NIB} , determined by the fraction of vertices with non-zero indegree (Theorem 1.7 in [ANNW26]). Specifically:

$$\alpha_{\text{NIB}} \geq \frac{n}{|\{v \in V | \text{indeg}(v) > 0\}|}$$

for $E \neq \emptyset$.⁸

The NIB can be easily calculated within linear time, whereas to compute the DUB, one has to determine the minimum valid α_{DUB} , which requires finding a subgraph $G' \subseteq G$ that maximizes its density $\delta(G')$.

2.6. Densest Subgraph Problem (DSP)

The problem of finding such a maximum density subgraph G' is known as the densest subgraph problem (DSP) and has been widely studied before. Goldberg provided the fastest exact algorithm in practice for this problem, narrowing it down to a binary search on Min-Cut/Max-Flow networks [Gol84]. While Max-Flow has been strictly polynomial for a long time, just recently Chen et al. found a solution that runs in almost linear time [CKL⁺22], drastically reducing the theoretical running time of exact DSP to $\mathcal{O}(|E|^{1+o(1)})$.

Even though this was a major breakthrough, DSP has proven to be easy to approximate in linear time. The most common approach was published by Charikar in 2000 [Cha00]: To determine a high-density subgraph, it is intuitive to iteratively remove the node with the lowest degree. Therefore for a given graph $G = (V, E)$, the algorithm works as follows:

- Initialize subset of vertices $S = V$ and set $S_{\text{max}} = S$, $\delta_{\text{max}} = \delta(G(S))$, where $G(S)$ denotes the subgraph induced by $S \subseteq V$.
- While $S \neq \emptyset$ repeat the following steps:
 - Delete vertex $v = \text{argmin}_{v \in S}(\text{deg}_{G(S)}(v))$, the vertex with the currently lowest degree, and remove it from the current set: $S = S \setminus \{v\}$.
 - If $\delta(G(S)) > \delta_{\text{max}}$: $S_{\text{max}} = S$ and $\delta_{\text{max}} = \delta(G(S))$
- return δ_{max} and $G(S_{\text{max}})$, the subgraph induced by S_{max} .

⁷Note that, by representing the average indegree of a directed graph or half the degree of an undirected graph, this definition differs from the usual normalized graph density.

⁸Otherwise α_{NIB} can be chosen arbitrarily high, because $H(G) = 0$ for any graph with $|E| = 0$.

This peeling approach runs in $\mathcal{O}(|V| + |E|)$ and guarantees a $1/2$ -approximation, but already yields significantly better results in practice than the worst case suggests [BGP⁺19].

However, the most practical algorithm chosen for this work is the later refinement of Charikar’s greedy algorithm by Boob et al. [BGP⁺19]. Greedy++ extends Charikar’s peeling approach throughout multiple iterations. Dependent on a node’s degree (when removed) in previous iterations, it updates a cumulative weight for each node, penalizing nodes that have been chosen early in recent iterations.

Empirical experiments showed that this in practice converged quickly, rarely taking more than 100 iterations [BGP⁺19]. It was also proven to always converge to the optimal solution [CQT22].

2.7. Maximum Planar Subgraph Problem (MPS)

Since extracting edges in form of a tree seems to be an effective strategy, as the evaluation in Section 4.3.1 confirms, one might also consider trying to extract more than $n - 1$ edges and store them in a low-cost-per-edge environment. The four main considerations for an expanded extraction approach are:

1. There needs to be a low bits-per-edge representation.
2. The structure must allow for efficient query support.
3. A computationally feasible way to extract such a structure must be given.
4. It must yield a natural and deterministic relabeling of the nodes.

Looking at the first two points, a planar graph⁹ appears as a natural candidate. A planar graph can be stored in a variety of ways, very much depending on the required query speed as well as its density. A simple approach was introduced by Turán in 1984 [Tur84], which uses 4 bits per edge [FFG⁺20].¹⁰ This unfortunately does not allow for quick queries on the graph [Jac89], which makes it a suboptimal choice for our use case. There are more specialized representations with more constraints as well, which we will be coming back to once we know more about the actual extracted structure.

Therefore, we now need to find a way to extract a planar subgraph, ideally the maximum planar subgraph (MPS). This problem is well known to be NP-hard, therefore, an exact algorithm is not feasible. However, we can use a simple greedy algorithm (GT, standing for Greedy-Triangles [CU24]) of Calinescu et al. [CFFK98], which, for a simple undirected graph $G = (V, E)$, works as follows:

- Initialize empty set of edges $E' = \emptyset$

⁹A graph is planar if and only if it is drawable in a 2-Dimensional space, without any of its edges crossing.

¹⁰Can be verified by looking within Turán’s proof [Tur84].

- **Phase A:** Until no more exist, find triangles in the original graph G . Add the edges to E' only if all three vertices belong to different connected components in the induced subgraph $G[E']$.¹¹
- **Phase B:** Once no triangles are left to add, iterate through the remaining edges and add the edge (u, v) to E' if both u and v belong to different components in $G[E']$ again.
- Finally, $G' = G[E']$ as the extracted planar subgraph.

This approximation was introduced in 1996 and refined in 1998 by Calinescu et al. [CFFK98] and shown to guarantee a $7/18$ -approximation in linear time (for bounded degree graphs). This is strictly better than the $1/3$ -approximation that a spanning tree implicitly delivers in linear time. The same publication also proposed a $4/9$ -approximation algorithm. But this only runs in polynomial time, and one has to compute a Graphical Matroid Parity solution, which introduces significant implementation, as well as computational effort.

Entropy Minimization: We adapt the GT algorithm specifically to minimize entropy by controlling the order in which triangles are processed in Phase A. Therefore, similar to MINETREX, each triangle was assigned a static weight and then chosen based on this ordering (ascending). For MINETREX, due to the monotonicity, one could simply substitute $\text{indeg}(j)$ as a weight for edge (i, j) , rather than the approximated actual cost, which is $\log_2(\text{indeg}(j))$. But now that we look at a complete triangle's weight, taking the sum of all three nodes' indegrees fails to accurately represent the expected cost. Therefore, for each triangle Δ in the graph, the weight was calculated to be:

$$\text{weight}(\Delta) = \sum_{(u,v) \in \Delta} \log_2(\text{indeg}(v))$$

Representation of Extracted Graph: Calinescu et al. show that the resulting graph is a triangular structure, which means that any cycle must be a triangle [CFFK98]. And if now two triangles shared an edge, this would result in a 4-cycle, violating that property. Therefore, triangles can share at most one vertex, which already perfectly matches the definition of a cactus graph, where two simple cycles can have at most one vertex in common. Hence, the resulting graph G' is always a triangulated cactus graph.

Other than that, a block graph, by its forbidden graph characterization, is a graph that does not contain the diamond graph¹² nor a cycle of length greater than or equal to four as an induced subgraph. It is easy to see that we cannot have a diamond graph, because this would violate that no two triangles share an edge. Also, no cycle of length greater than three can exist, because in phase A, strictly triangles were added, and then in phase B, we limit ourselves to adding edges that connect disjoint components, not adding any

¹¹For $E' \subseteq E$ we denote the spanning subgraph (V, E') of G by $G[E']$. Note that this is different from the vertex-induced subgraph $G(S)$ from section 2.6, because $G[E']$ is induced by the edges and also always contains the complete set of original nodes.

¹²A diamond graph simply is a K_4 with one missing edge.

new cycles. Since neither of the structures can even occur as a general subgraph, they in particular do not occur as an induced subgraph either, therefore, the structure extracted by the GT algorithm is also a block graph.

Building on this, Chakraborty et al. introduced a way to store unlabeled, undirected block cactus graphs within $2.092n + o(n)$ bits [CJSS24]. A block cactus graph is defined as a graph in which every biconnected component is either a clique or a cycle. Block graphs as well as cactus graphs are simply special cases of this more general class, meaning that, since our graph is part of both classes, it is also in particular a block cactus graph. However, this still leaves room for future optimization if one were to further optimize the representation to fit the intersection of both structures.

While their structure stores it efficiently, it also supports `adjacent( $u, v$ )` and `degree( $u$ )` in constant time, as well as `neighborhood( $u$ )`, which returns the set of neighbors in a deterministic ordering in $\mathcal{O}(\text{degree}(u))$. Specifically, because querying the i -th neighbor of a vertex in constant time is highly beneficial for TREX navigation, this linear time bound could be a potential bottleneck.

Also, the fourth requirement of a canonical relabeling scheme remains a challenge. Chakraborty et al.'s representation already transforms the cactus graph into a Tree with additional dummy nodes, in principle yielding a canonical labeling of the nodes. However, unlike TREX's single dummy node, this resulting tree contains plenty of dummy nodes, which are spread throughout the tree. Therefore, excluding them on the fly is non-trivial and might require an additional bitvector to store the "dummy" property, or introduce slower query time.

Other than that, a further caveat remaining — once a labeling scheme is settled — is that this is an undirected planar subgraph, meaning that, for directed graphs, we again have to store one bit per edge to determine its direction. But because a cactus graph is stored, it can hold at most $\lfloor \frac{3(n-1)}{2} \rfloor$ edges,¹³ resulting in a total space of at most $3.592n$ or $2.092n + m_P$, where m_P is the number of edges in the planar subgraph.

The 4/9-approximation from Calinescu et al. [CFFK98] produces a maximal triangulated cactus subgraph [CU24] and could thereby be stored within the same succinct representation [CJSS24]. If one wanted to utilize an even closer approximation, a new representation would likely be needed as well.

3. Implementation

3.1. Architecture and Graph Construction

In the area of succinct data structures, one distinguishes between theoretical design and practical engineering. Succinct representations are usually implemented in low-level languages to gain access to manual memory allocation as well as some hardware-specific operations (e.g. Popcount). While this approach is crucial for industry-level

¹³Achieved by storing as many triangles as possible, while only sharing at most one vertex (for example, see [Wes01]).

implementations, it introduces unwanted complexity in the testing and evaluation process. Therefore, this thesis completely focuses on the conceptual design and empirical evaluation of the different concepts. The complete repository, including the code and the benchmark graphs, is made public separately.¹⁴

Dummy Implementations: The representation of a TREX compressed graph relies on three basic succinct structures: bitvectors, LOUDS, and wavelet trees. All of which have been implemented as simple Python classes that do not support constant-time queries and are not compact. The dummy bitvector class is also largely used in the dummy LOUDS as well as wavelet tree implementations. If one were to only replace the bitvector with a succinct implementation, this would already make LOUDS fully succinct and drastically improve space and query time for wavelet trees as well. It is important to note that in contrast to most standard literature, bitvectors were implemented zero-based. This leads to all the queries being index-shifted compared to book implementations or also the TREX queries proposed by Alaoui et al. [ANNW26]. Still, this should be no problem because most succinct Python libraries and C++ bindings are already zero-based.

The LOUDS implementation also slightly generalizes the definition of the paper, where directed graphs were assumed. This was done as follows: Each LOUDS instance already has a dummy node (meaning 10) at the beginning. It can easily be made a super-root, whose children are simply extended to fit the number of connected components to obtain the ability to store forests as well. Only some operations must then be slightly tweaked to ensure the dummy node is never accounted for in any queries. Those small changes, however, are just simple if statements and index shifts not impacting the $\mathcal{O}(1)$ in any way.

Graph Construction and Evaluation: A dedicated builder class was implemented to construct the TREX representation of a given NetworkX graph. For directed graphs, a minimum spanning tree of each connected component is created, using the target node's indegree – or minimum indegree of reciprocal edge pairs – as weights. Those trees are then unified via one LOUDS forest, enabling a BFS for canonical relabeling of the nodes. And within one edge pass in BFS order, the direction bitvector D is populated and the concerning edges of the forest are extracted from a copy of the original graph. Now only one pass over the residual graph is necessary to construct the wavelet tree A' as well as the boundary bitvector S' .

For the undirected case, edges are first greedily directed, after which construction proceeds identically to the directed case. Only now the bitvector D is omitted, and no reciprocal edges can exist. Finally, an evaluation pipeline was implemented, which takes a TREX instance (directed or undirected) and computes the empirical metrics – including the theoretical figures for planar extraction – later needed for the evaluation.

¹⁴https://github.com/Laserjullu/trex_toy

3.2. Navigational Logic and Undirected Generalization

TREX navigation: On top of those implemented dummy structures, the navigational logic had to be implemented for a given TREX instance, assembled by the builder. While implementing those methods, according to the explained queries in the paper by Alaoui et al. [ANNW26], a few flaws occurred — as is natural when translating a theoretical specification into an actual implementation. Let the TREX representation of a directed graph $G = (V, E)$ consist of (T, A', S', D) , with T being the extracted forest represented using LOUDS, A' the residual graph's adjacency string as a wavelet tree, S' the bitvector marking the boundaries in A' , and D serving as the indicator of edge directions in the LOUDS forest, where $D[v - 1] = 0$ if and only if the removed original edge between v and $\text{parent}(v)$ points towards the parent.

Note that, as in the accompanying implementation, all operations are described zero-based; therefore, they can also slightly vary from the form introduced by Alaoui et al. [ANNW26].

- D : Throughout most TREX queries, the direction bitvector D is used inversely to its definition in the paper [ANNW26]. Therefore, for example, counting v 's ingoing edges from its children requires rank_0 on $D[c_1 - 1, \dots, c_k - 1]$ rather than rank_1 . The analogous inversion appears on every occurrence of **rank** and **select** queries on D throughout. The direct access to check whether a node points towards the parent, however, was always done correctly.
- $\text{inneighbor}_v(i)$ determines the i -th inneighbor of a node v , where Tree nodes are considered before residual graph edges. The query's original implementation contained a logical mismatch when querying the residual graph [ANNW26]. To find the desired edge in A' , the algorithm correctly determines its zero-based index as $y = \text{select}_v(A', i - d_T)$, where d_T denotes the indegree of v in T . However, it is then attempted to compute the originating vertex of this edge as $w = \text{rank}_1(S', y)$. This mistakenly feeds an index stemming from A' directly into S' , to correct this, the edge index must first be mapped to its corresponding 0 in S' . The corrected formula for w as the i 'th inneighbor of node v therefore is:

$$w = \text{rank}_1(S', \text{select}_0(S', y + 1))$$

We take the TREX instance shown in Figure 3 as an example, on which we compute $\text{inneighbor}_8(2)$. For additional detail on the individual steps, see the provided pseudocode (Algorithm 1).

Node 8 in T has an indegree of 1, meaning its second inneighbor must lie in A' . We find the first occurrence of 8 in A' via $\text{select}_8(A', 1)$ (first edge pointing towards 8), which returns $y = 2$. The original formula would now incorrectly return $\text{rank}_1(S', 2)$, giving us node 2. The corrected logic now first maps this index with $\text{select}_0(S', 2 + 1)$ to index 6 in S' , now leading to $\text{rank}_1(S', 6)$ correctly identifying node 4 as the second inneighbor of node 8.

- The `adjacent(u, v)` query mistakenly bounds its search interval within A' using the total `outdegree(u)` rather than the residual degree, which is calculated as:

$$\text{outdegree}(A', u) = \text{select}_1(S', u + 1) - \text{select}_1(S', u) - 1$$

Additionally in the same query `rank1(A', s)` was calculated instead of the necessary `rankv(A', s)`. The interval of the outneighbor-list was also cut short by one, by only accounting for characters up until `outdegree(u) - 1`.¹⁵

- In the `outneighbor_rank(v, w)` query, when w is a child of v within T and the edge between them points from v to w , the paper subtracts one instead of adding one. By our ordering, an outgoing parent-edge always occupies index 1, therefore shifts the rank of each out-child by one, meaning `rank = j - [D[v - 1] = 0]` should read `rank = j + [D[v - 1] = 0]`.

Furthermore, a silent off-by-one discrepancy occurred because the baseline rank r on A' was evaluated up to $s - 1$ instead of s , excluding a potential occurrence of w . The identical off-by-one discrepancy applies to `inneighbor_rank`.

Undirected Graphs: In contrast to the directed case, undirected graph queries were not yet described extensively [ANNW26]. However, the undirected queries mostly mirror the behavior of the directed ones, only with some different definitions which must be assumed. In this work, only the following operations were implemented:

- `degree(v)`: We simply sum over the indegree and outdegree of v within A' as well as the degree within T , which is now natively supported because we do not care about the direction of the edges.¹⁶
- `adjacent(u, v)`: Completely symmetric to the directed case.
- `neighbor(v, i)`: Returns the i -th neighbor of v , where first all the tree neighbors are considered, followed by the "outgoing" and then "ingoing" edges within A' .
- `neighbor_rank(v, w)`: Returns the i such that `neighbor(v, i) = w`. Therefore, the same ordering from `neighbor(v, i)` is held, and a similar logic as in `inneighbor_rank` and `outneighbor_rank` for the directed graphs is applied.

To verify the logic, each navigational query was validated against NetworkX ground truth on more than 1000 randomly generated directed and undirected graphs, including manually constructed edge cases, with no discrepancies found. For reference, the complete Python implementations of all directed as well as undirected navigational queries — adapted to use zero-based indexing throughout — are listed in full in appendices A and B.

¹⁵We assume that the outdegree within A' was meant rather than the general outdegree.

¹⁶Only one must potentially be added to the number of children, such that the node from/to parent is also considered.

Algorithm 1: Algorithm for the `inneighbor(v, i)` operation on a graph using the TREX representation.

Input: A vertex v , an integer i and a TREX instance of a directed graph G , consisting of: The LOUDS tree T , according direction bitvector D , residual adjacency wavelet tree A' and the corresponding bitvector S' marking the outneighborhood boundaries.

Output: If existing, the ID of the i -th inneighbor of vertex v , -1 otherwise.

```

1 if  $\text{indegree}(v) < i$  or  $i < 1$  then
2   return  $-1$ 
3  $\text{has\_ingoing\_parent} \leftarrow [\text{access}(D, v - 1) = 1 \text{ and } \text{parent}(T, v) \neq 0]$ ;
4 if  $i = 1$  and  $\text{has\_ingoing\_parent}$  then
5   return  $T.\text{parent}(v)$ ;
6  $T_{\text{indegree}} \leftarrow 0$ ;
7  $T_{\text{indegree}} \leftarrow T_{\text{indegree}} + [\text{has\_ingoing\_parent}]$ ;
8 if  $\text{degree}(T, v) > 0$  then
9    $\text{last\_child} \leftarrow T.\text{child}(v, \text{degree}(T, v))$ ;
10   $\text{first\_child} \leftarrow T.\text{child}(v, 1)$ ;
11   $T_{\text{indegree}} \leftarrow T_{\text{indegree}} + \text{rank}_0(D, \text{last\_child} - 1) - \text{rank}_0(D, \text{first\_child} - 2)$ ;
12 if  $i \leq T_{\text{indegree}}$  and  $\text{degree}(T, v) \neq 0$  then
13    $j \leftarrow i - [\text{has\_ingoing\_parent}]$ ;
14    $c_1 \leftarrow T.\text{child}(v, 1)$ ;
15    $o \leftarrow \text{rank}_0(D, c_1 - 2)$ ;
16   return  $\text{select}_0(D, j + o) + 1$ ;
17  $j \leftarrow i - T_{\text{indegree}}$ ;
18  $y \leftarrow \text{select}_v(A', j)$ ;
19  $a \leftarrow \text{select}_0(S', y + 1)$ ;
20 return  $\text{rank}_1(S', a)$ ;
```

4. Empirical Evaluation

4.1. Methodology

The target of this evaluation is not to determine the memory consumption of the Python implementation directly, but rather to evaluate based on the space estimations of the underlying structures, introduced in earlier sections. For each test graph $G = (V, E)$, several representations were compared against each other to closer isolate the source of compression gains. For the undirected as well as the directed case, the absolute baseline is chosen to be an adjacency list-like representation. The adjacency lists are concatenated and stored as an array A ($m \lceil \log_2 n \rceil$ bits), while a second array S holds the start indices for each vertex's neighborhood within A ($n \lceil \log_2 m \rceil$ bits).¹⁷

Beyond this baseline, we consider more sophisticated and closer to real-world representations. For a directed graph, we compute the space in the same way using A and S , but now compress both using a wavelet tree as well as a bitvector. This mirrors exactly the final TREX representation, only without the tree removal step (how the residual graph $G - T$ is stored). This gives us a total space of $H(G) + \log_2 \binom{n+m}{n}$; let the space of a graph with this representation be called $\omega(G)$.

For undirected graphs, however, we make some finer distinctions. Due to the possibility of bidirectional adjacency navigation of wavelet trees and LOUDS, it is not necessary to store every edge twice. Additionally, we are specifically interested in the performance of the greedy edge-directing preprocessing of U-MINETREX. To further quantify its impact, we calculate multiple edge orientation strategies. Each of the following approaches is a possible way to keep only one of the directions of an undirected edge $\{u, v\}$, directing the graph while reducing the edge count to m , before computing $\omega(G)$:

- **GREEDY:** We use the greedy approach of U-MINETREX, which attempts to minimize the entropy of the resulting adjacency string [ANNW26]. To do so, an edge is directed to the vertex with the higher degree, effectively deleting the edge directed towards $\text{argmin}(\deg(v), \deg(u))$.
- **LEXICOGRAPHICAL:** We direct edges solely based on the lexicographical order of their original vertex IDs. Keeping an edge (u, v) if and only if $u > v$. This serves as a deterministic baseline one could come up with if no thought went into the resulting adjacency string's entropy.
- **RANDOM:** For each edge, we independently keep one of the two directions with equal probability.
- **ANTI-GREEDY:** We want to artificially increase the entropy by always picking the edge that greedy did not pick. And even though this is far from a worst case, it tries to provide a picture of how big the impact of an orientation strategy can be.

¹⁷For undirected graphs, we count each edge twice.

The random edge orientation described above — as well as the random spanning tree strategy discussed in Section 4.3.3 — were generated using a fixed random seed of 170826, ensuring the reported figures are exactly reproducible.

Two main representations whose space consumption we are most interested in are the ones of the TREX- as well as PLEX-reduced graphs. They are first described for the directed case before explaining the adjustments for undirected graphs below. For TREX, we store it as explained in section 2.4, using LOUDS plus a direction bitvector for the extracted tree, and compute $\omega(G - T)$ for the residual graph, resulting in a total space of $2n + n + H(G - T) + \log_2 \binom{m+1}{n}$.

For PLEX, we store the residual graph identical to TREX. Only for the removed subgraph, we use Chakraborty et al.’s structure, taking $2.092n$ bits for the extracted cactus graph and m_P for the direction bitvector, where m_P is the number of extracted edges. This gives us a total space of $2.092n + m_P + H(G - G_P) + \log_2 \binom{m-m_P+n}{n}$, where G_P is the extracted planar (cactus) subgraph.

For undirected graphs, we compute the TREX and PLEX representation metrics based on the directed graph obtained after the greedy U-MINETREX orientation step, which is also precisely the graph from which the actual tree is removed in U-MINETREX and therefore the bounds DUB and NIB are calculated from. Only now is the space for the, in the undirected case needless, direction bitvectors omitted for both TREX and PLEX.

4.2. Datasets

Each graph’s original representation was manually modified to an edge list and then parsed into a created evaluation pipeline. The edge lists — including those of the created synthetic graphs — are publicly available on GitHub.¹⁸ Due to the extensive memory usage of NetworkX, only graphs up to a limited size were feasible on the testing machine’s limited 16 GB of RAM.

To ensure a diverse evaluation, 11 graphs of different domains were chosen. A short overview of their respective size and density is shown in Table 1. Let $G = (V, E)$ be a directed/undirected graph:

- **Directed Real-World Networks:**

- AMAZON0302 [LK14, LAH07]: A product co-purchasing network, collected from Amazon in 2003. A directed edge $(u, v) \in E$ exists if and only if product v is frequently listed in product u ’s "Customers Who Bought This Item Also Bought" list. It contains dense neighborhoods and a natural bound for the node’s outdegrees.
- CIT-HEPPH [LK14, LKF05, GKG03]: A citation network of all the high-energy physics phenomenology papers published from 1993 to 2003. A directed edge $(u, v) \in E$ indicates a citation of paper v within paper u . Due to its temporal nature, it is naturally close to acyclic.

¹⁸https://github.com/Laserjullu/trex_toy

- SLASHDOT0811 [LK14, LLD09]: A social network of Slashdot, where edges (u, v) represent that user u has tagged user v as a friend or foe. It embodies a classical power-law degree distribution.
 - WIKILINK_GER_2005 [CLM19]: A hyperlink network of the German Wikipedia in 2005. An edge $(u, v) \in E$ exists if and only if article u contains a hyperlink to article v .
 - WIKI-VOTE [LK14, LHK10b, LHK10a]: Covers all Wikipedia administrator elections up until January 2008. An edge $(u, v) \in E$ means that u has cast a vote for user v . It was chosen for its significant number of zero indegree nodes and its possibility of users with identical voting patterns.
- **Undirected Real-World Networks:**
 - ROADNET-PA [LK14, LLD09]: The road network of Pennsylvania where vertices represent intersections or endpoints and edges are the roads connecting them. By being very sparse, near planar, and highly local, it provides a strong structural contrast to social networks.
 - BIO-GRID-YEAST [RA15]: A biological protein-protein interaction network, where edges represent interactions between different proteins.
 - TWITCH GAMERS [LK14, RS21]: A snapshot of the largest connected component of the social network Twitch in 2018. It is based on mutual followship and provides a dataset with a massive degree skew in an undirected environment.
 - **Synthetic Graphs (Undirected):**
 - ERDŐS-RÉNYI: Was generated with NetworkX’s implementation of the Erdős-Rényi-Gilbert model $G(n, p)$ [Gil59, ER59]. One assigns a deterministic number of nodes n as well as a probability p . Then each possible edge is included with independent probability p . Because of its independent random creation, it is expected to have high entropy, serving as a stress test for Tree Extraction and graph compression in general. For the creation of the test graph used in this evaluation, p was set to 0.005.
 - BARABÁSI-ALBERT [BA99]: Created using the implementation of NetworkX; generates a graph of n nodes by sequentially adding nodes u with an outdegree of 14 each, where the probability of edge (u, v) being added when node u is added is proportional to $\text{degree}(v)$. This preferential attachment breaks the independence of edges and creates a more realistic uneven degree distribution.
 - BIPARTITE: A complete and symmetric bipartite graph $K_{1000,1000}$ was included as well. For two partitions of the 2000 nodes U and V both of size 1000, each possible edge $(u, v) \in U \times V$ is included. This provides a very dense, triangle-free stress test.

Additionally, seven further graphs of varying topologies were picked and exclusively used as validation sample to test the generalizability of the compression-potential heuris-

tic introduced in Section 4.3.4: EMAIL-EU-CORE, P2P-GNUTELLA24, CA-ASTROPH, FACEBOOK_COMBINED and OREGON1_010331 from SNAP [LK14], POWER-US-GRID and CELEGANSNEURAL from Network Repository [RA15].

Table 1: Overview of benchmark graph datasets.

Dataset	type	n	m	density ($\frac{m}{n}$)
AMAZON0302	directed	262,111	1,234,877	4.71
CIT-HEPPH	directed	34,546	421,578	12.20
SLASHDOT0811	directed	77,360	905,468	11.70
WIKILINK_GER_2005	directed	244,112	3,659,390	14.99
WIKI-VOTE	directed	7,115	103,689	14.57
ROADNET-PA	undirected	1,088,092	1,541,899	1.42
BIO-GRID-YEAST	undirected	6,008	156,945	26.12
TWITCH GAMERS	undirected	168,114	6,797,557	40.43
ERDŐS-RÉNYI	undirected	20,000	999,036	49.95
BARABÁSI-ALBERT	undirected	40,000	559,804	14.00
BIPARTITE	undirected	2,000	1,000,000	500.00

4.3. Results and Discussion

4.3.1. Base Compression Performance

Directed Graphs: Across all the directed networks, a consistent hierarchy was seen, where TREX always reduced the space requirement compared to the baseline wavelet tree and bitvector representation $\omega(G)$, yielding average savings of 10.08%. Furthermore, extracting a planar/cactus subgraph improved the savings by an additional 3.40% on average, relative to TREX’s space consumption. A brief overview of the key metrics is provided in Table 2.

Undirected Real-World Graphs: For undirected graphs, additional space savings already arise naturally, coming from directing the edges rather than storing them twice. Therefore, as a baseline for fair comparisons, $\omega(G_{\text{GREEDY}})$ was taken, where G_{GREEDY} is the graph whose edges are oriented towards the higher degree node. And as already expected, the space usage roughly halves simply by applying this preprocessing step, although it is still dependent on the indegree distribution and the non-linear scaling of the bitvector S .

Additionally, because in the chosen baseline, edges are already directed towards higher degree nodes, one might expect lower relative savings from TREX. However, TREX provided measurable compression across all three real-world datasets. While comparatively modest on the denser graphs BIO-GRID-YEAST and TWITCH GAMERS with 4.35% and 3.17%, ROADNET-PA showed savings of a staggering 64.00%. The reason for this immediately lies in the fact that $m \approx n$ on this graph; consequently, more than

two thirds of the edges can be represented within the extracted forest, resulting in a 2 bits-per-edge representation compared to the 20.89 bits required to store the residual edges.¹⁹ Even though PLEX could only extract less than 6% additional edges, those had, coming from the small density, a large impact, by saving an additional 10.11% relative to TREX’s space consumption. On BIO-GRID-YEAST and TWITCH GAMERS planar extraction – while being able to extract roughly 35% more edges than a forest on both – had a comparatively modest impact, saving an additional 1.46% and 1.01%.

Synthetic Graphs: On the synthetic graphs, which are undirected as well — therefore use the same greedy baseline — we can see quite varying results; both ERDŐS-RÉNYI and BIPARTITE did not show a large improvement (1.84% and 0.19%), whereas BARABÁSI-ALBERT’s TREX representation improved space consumption by 7.56%. PLEX also showed a negligible further improvement of 0.57% on average.

Table 2: Core compression results: All the space usages are displayed as bits per edge. Percentage changes are relative to $\omega(G)$ for directed and $\omega(G_{\text{GREEDY}})$ for undirected graphs. "Array" includes the array for the adjacency string as well as the array marking neighborhood borders.

Dataset	Array	$\omega(G)$	$\omega(G_{\text{GREEDY}})$	TREX	PLEX	ΔTREX	ΔPLEX
AMAZON0302	22.46	18.28	-	14.68	13.27	19.69%	27.42%
CIT-HEPPH	17.56	14.04	-	12.87	12.43	8.36%	11.47%
SLASHDOT0811	18.71	14.94	-	13.61	13.50	8.92%	9.61%
WIKILINK_GER_2005	19.47	15.00	-	13.83	13.54	7.79%	9.71%
WIKI-VOTE	14.17	11.03	-	10.41	10.29	5.65%	6.74%
ROADNET-PA	57.53	42.13	21.00	7.56	6.80	64.00%	67.64%
BIO-GRID-YEAST	26.73	23.44	10.65	10.19	10.04	4.35%	5.75%
TWITCH GAMERS	36.59	31.63	13.93	13.49	13.35	3.17%	4.15%
ERDŐS-RÉNYI	30.42	28.72	14.06	13.80	13.66	1.84%	2.81%
BARABÁSI-ALBERT	33.50	30.14	14.13	13.06	12.97	7.56%	8.23%
BIPARTITE	22.04	21.95	9.99	9.97	9.97	0.19%	0.19%

4.3.2. Impact of Edge Orientation

Across all undirected benchmark datasets, the four different edge orientation heuristics were evaluated to measure the impact and performance of the U-MINETREX edge orientation approximation, see Figure 4. GREEDY provided the best results throughout all six datasets. Looking specifically at the three real-world datasets, we see a strict hierarchy one could expect across the remaining strategies as well: ANTI-GREEDY

¹⁹A complete set of all used metrics, including ones not shown in any tables, is available as a CSV file on the GitHub repository: https://github.com/Laserjullu/trex_toy

consistently performed worst, and LEXICOGRAPHICAL was always slightly better than the truly random orientation strategy.

Generally, a natural first guess would be that LEXICOGRAPHICAL is practically random, therefore, the results should not noticeably vary from RANDOM. This turns out to be misleading, because even though the choice is random for each edge individually, the choices are not independent — leading to nodes with higher IDs systematically being targeted less often. Additionally, the original IDs might not have been chosen at random to begin with, but this was not evaluated against the datasets used in this work. Together, this explains why LEXICOGRAPHICAL outperformed RANDOM throughout all six graphs.

However, when looking at the synthetic graphs specifically, such strict behavior is not observed anymore. For example, on BIPARTITE we notice that ANTI-GREEDY performed better than both RANDOM and LEXICOGRAPHICAL. For RANDOM, this is obvious, because it creates a roughly uniform indegree distribution, compared to the highly skewed one from either greedy.²⁰ Only for the lexicographical part, because each node in the first partition is naturally assigned a "smaller" ID than the one in the second [Deva], you expect all the edges to be pointed towards the nodes in the first partition. But because we are doing lexicographical comparisons rather than numerical, there is no straight relation between those two partitions (e.g. "2" > "1001" but $1001 > 2$).

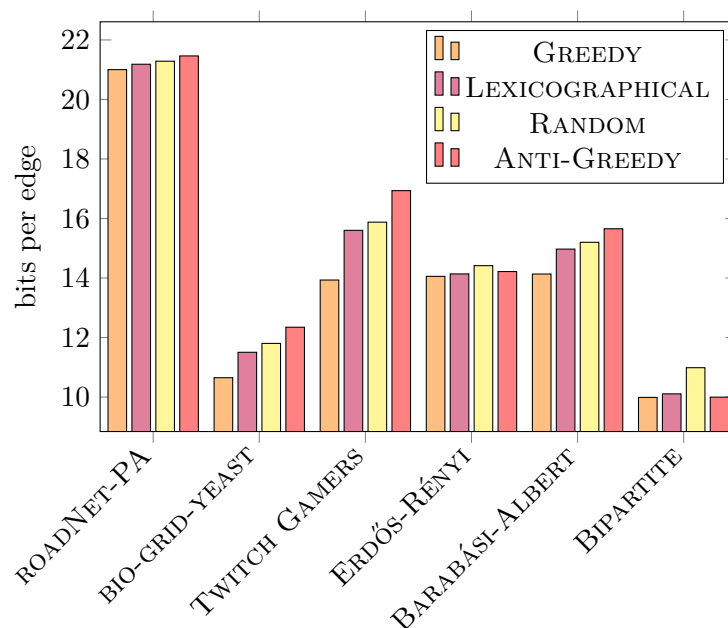


Figure 4: Bits-per-edge comparison of the different edge orientation strategies

²⁰Note that on $K_{1000,1000}$ each vertex has an identical degree of 1000, therefore the resulting skew results entirely from the deterministic tie-breaking of the implementation, rather than from an actual degree-based decision.

4.3.3. Greedy MINETREX approximation

The core foundation of MINETREX relies on finding a spanning tree T , which minimizes the residual empirical entropy $H(G - T)$. While the greedy algorithm provides an optimal additive linear error approximation to the problem [ANNW26], its practical relevance on real-world networks is not yet quantified. For isolation of the greedy extraction's impact, it was benchmarked against two other approaches:

- **RANDOM-TREE:** Picking a truly random spanning tree by assigning a random weight to each edge and then determining the corresponding minimum spanning tree.²¹
- **ANTI-GREEDY-TREE:** Approximating the worst-case entropy by determining a maximum cost spanning tree, using the same weighting scheme from U-MINETREX, but maximized rather than minimized.

To ensure easier comparability and interpretation, $\Delta(TREX_{\text{RANDOM-TREE}})$ and $\Delta(TREX_{\text{ANTI-GREEDY-TREE}})$ were calculated, completely analogously to $\Delta(TREX)$ only now of course with a different residual graph.

And as it turned out, both strategies did not perform fundamentally different across all networks (Figure 5). The greedy MINETREX algorithm showed savings of 11.96% on average across all 11 networks, compared to the 10.87% and 10.01% of RANDOM-TREE and ANTI-GREEDY-TREE. With less than a two percent difference between the approximated best and worst case, the room for optimization appears fairly small. Though on specific graphs the differences were more substantial, like on BIO-GRID-YEAST, where we saw 4.35%, 3.22% and 2.17%, approximately halving the space savings from the greedy strategy to ANTI-GREEDY-TREE. Nonetheless, a clear tendency is notable that the primary savings achieved by TREX are mainly driven by the edge extraction itself, rather than the specific strategy to optimize residual entropy.

This immediately suggests that the overall compression potential is largely determined by the fraction of edges, which we are able to store in a low bits-per-edge environment. And because we extract a single spanning tree/forest, the number of extracted edges is always roughly $n - 1 \approx n$, meaning $\approx \frac{n}{m}$ of the m edges are extracted. Therefore a simple assumption could be made that the relative space savings in percent p scale linearly with $\frac{n}{m}$, giving us $p \approx c \cdot \frac{n}{m} \iff p \approx c \cdot \delta(G)^{-1} \iff \log_2 p \approx \log_2 c - \log_2(\delta(G))$.

4.3.4. Compression-Potential Heuristic

Following immediately from the insights above, a linear model was fit to the logarithmic density as well as the logarithmic relative space savings of the 11 benchmark datasets, where:

$$\log_2 p = \beta_0 + \beta_1 \log_2 \delta(G) + \epsilon$$

Where p is again the relative space-saving in percentage points. The parameters β_0 and β_1 were determined to be 6.6074 and -0.9869 . β_1 's 95% confidence interval was also

²¹For bidirected edges in directed graphs, the deleted orientation is also determined by coin flip.

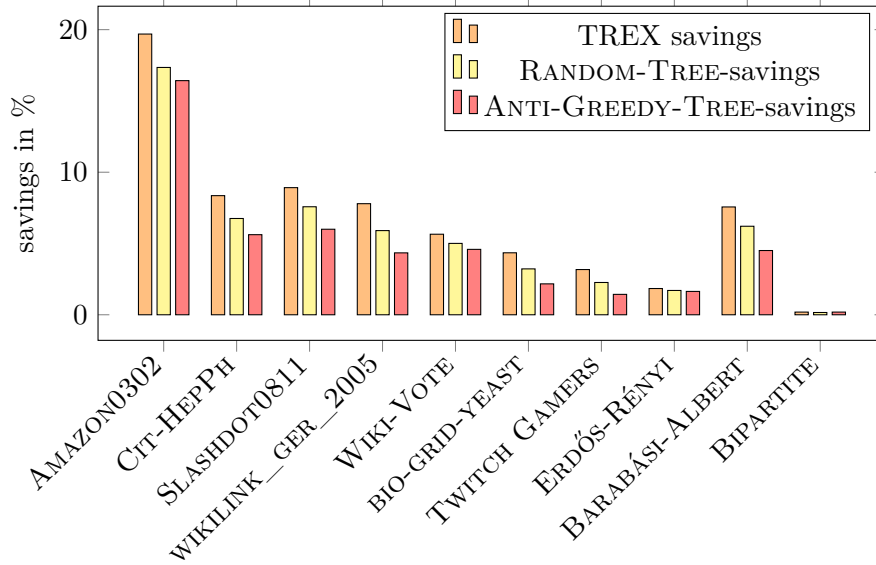


Figure 5: Relative space savings across spanning Tree Extraction strategies. Note that ROADNET-PA was omitted due to scaling reasons. It showed savings of 64%, 63.44% and 63.23% (TREX-savings, RANDOM-TREE-savings, ANTI-GREEDY-TREE-savings).

$[-1.053, -0.921]$, indicating that our earlier $\beta_1 := -1$ appears to have been a reasonable guess at the true slope. Also, this regression already fit the data well, resulting in an R^2 value of 0.992, indicating that the model already explains nearly all the logarithmic model’s variance. Even with such a low sample size, the p-value was ≤ 0.001 , standing for high statistical significance.

As a test sample to measure the model’s generalizability, the seven validation datasets introduced in Section 4.2 were used. On those predictions, we find a mean absolute error of 2.73 percentage points (after doing the inverse logarithmic transformation). This means our model generalized well, considering the low sample size and high variance in our target variable (ranging from $p_{true} = 3.97\%$ up to $p_{true} = 64.29\%$), as seen in Figure 6.

Even though those results sound impressive, we can not take them as too representative. To give a proper result, one would have to examine a much higher quantity of graphs. As a brief counterexample, where density alone is a bad predictor, two graphs with identical density of 2.997 ($n = 1000$, $m = 2997$) were created. G_A has two hubs – both having ingoing edges from each of the 999 other nodes – and a simple cycle of length 999.²² G_B has 3 hubs with each 999 ingoing nodes, see Figure 7 for rough visualization.

²²The cycle only adds 998 edges in practice, because one edge to a hub was already included, no matter how the cycle is chosen. Therefore, a single chord of the cycle – connecting two arbitrary cycle vertices – was added to exactly match density with G_B

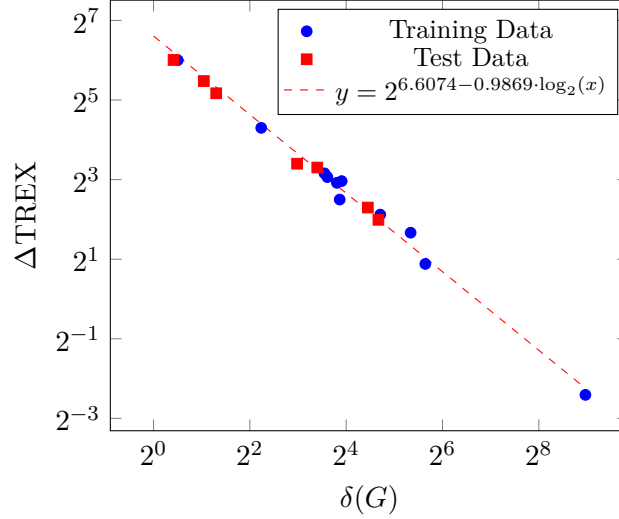


Figure 6: Relative space savings against density $\delta(G)$. ΔTREX again represents the relative space savings compared to the baseline ($\omega(G)$ for directed and $\omega(G_{\text{GREEDY}})$ for undirected graphs).

Even though those graphs have the same density, one can quickly tell that for G_B , there is not much hope to gain space, because all the edges are already pointed towards three hubs, containing very little entropy per edge already. On the other hand, G_A contains many nodes with indegree one, where large space savings can be expected. The numbers also verify this, on G_A we see an improvement of 56.76% compared to only 2.77% on G_B .

4.3.5. Theoretical Bounds

While the density heuristic provides a reliable empirical prediction of TREX compression potential, it is still just an empirical estimation. To safely choose a compression scheme upon compression, we need to ensure that TREX can not perform significantly worse than the heuristic suggests. A mathematically sound attempt at this are the bounds DUB and NIB introduced in Section 2.5. They give an estimation of $H(G - T)$, leaving out the overhead for varying bitvectors and the tree at first. Both of those upper bounds have, as they should, not been exceeded by any of the test graphs. Nonetheless, the upper bounds were also never really tight. Both bounds consistently kept overestimating the actual entropy by roughly $a \cdot n \log_2 n$, where $a \in [0.73, 1.26]$ for DUB, and $a \in [0.77, 1.08]$ for NIB (Table 3). On real-world graphs, the NIB always beat the DUB, except once on ROADNET-PA. Important to note is that intuitively on undirected graphs, an indegree is not really defined. Therefore, the NIB is computed based on the directed graph produced by the greedy orientation step of U-MINETREX. But because edges are systematically pointed towards nodes with high degree, many low-degree nodes are left with an indegree of zero, increasing α_{NIB} . Therefore, the calculated NIB values are still sound, but benefit

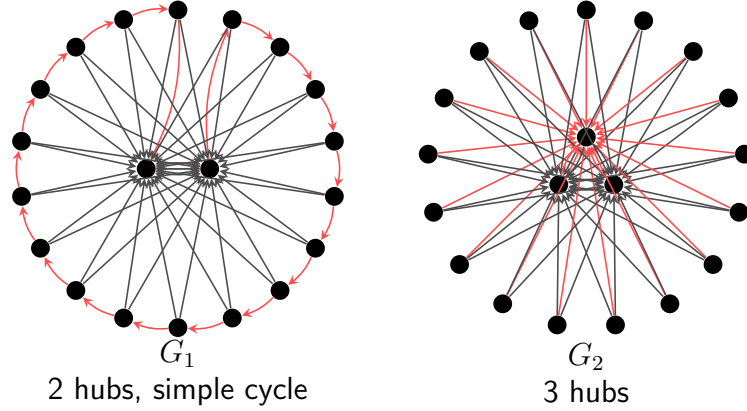


Figure 7: Visualization of graphs conceptually identical to G_A and G_B , only with $n = 20$ rather than 1000. Red edges form a possible removed spanning tree. Note how the residual structure (black edges) is essentially identical between the graphs, despite the large difference in their respective intrinsic information.

from the greedy edge orientation — unlike the DUB, which is determined by subgraph density and consequently unaffected by edge orientation.

Also noteworthy is that exact calculation of the DUB requires the maximum density subgraph, which was only approximated in this work. While the approximations were shown to work well on real-world graphs [BGP⁺19], it is only guaranteed to be a $1/2$ approximation [Cha00]. The implementation used was adapted from NetworkX’s Greedy++ implementation [Devb], only slightly modified to support multigraphs, which is identical to a directed graph, where reciprocal edges get a multiplicity of two. This, however, would not matter much for the comparison of the two bounds and would only worsen the tightness of the DUB even further, which was already inferior to the NIB on most real-world graphs.

Bounds as Predictors Beyond their tightness as bounds on $H(G - T)$, one could also ask how DUB and NIB performed if used directly as compression-potential predictors. To answer this, each bound was modified by adding the required space for the tree T , S' , as well as for the direction bitvector D (for directed graphs). Each resulting metric was then compared to the baseline representation, yielding average predicted savings of 0.79% (DUB) and 1.04% (NIB) on the 11 benchmark datasets — far below the 11.96% TREX was able to save on average. To compare the bounds’ predictive performance directly against the density heuristic’s, the same metrics were computed on the seven validation datasets. There, both bounds predicted a net loss on average (-5.60% and -3.48%), with mean absolute errors of 30.47 (DUB) and 28.35 (NIB) percentage points — drastically worse than the 2.73 percentage points of the density heuristic. This was mainly driven by POWER-US-GRID and OREGON1_010331, whose predictions each missed by more than 60 percentage points. Both graphs show a very low density, leading to an overestimation of roughly $n \log_2 n$ having a bigger relative impact, which also cannot be compensated

Table 3: Calculated bounds for each network. Note that for the calculation of the DUB, 13 Greedy++ iterations were used for graphs with $m \leq 1,000,000$ and one iteration otherwise. For simplicity, m' — as the residual edges — is approximated with $m - n$ rather than $m - (n - c)$, where c is the number of connected components.

Dataset	$\frac{H(G-T)}{m'}$	$\frac{DUB}{m'}$	$\frac{NIB}{m'}$	$\frac{DUB-H(G-T)}{n \log_2 n}$	$\frac{NIB-H(G-T)}{n \log_2 n}$
AMAZON0302	16.88	20.73	20.60	0.79	0.77
CIT-HEPPh	13.30	14.80	14.60	1.11	0.96
SLASHDOT0811	14.14	16.04	15.45	1.25	0.87
WIKILINK_GER_2005	14.23	15.83	15.50	1.26	0.99
WIKI-VOTE	10.57	11.55	11.53	1.04	1.02
ROADNET-PA	17.92	53.15	56.80	0.73	0.81
BIO-GRID-YEAST	10.27	10.85	10.81	1.15	1.08
TWITCH_GAMERS	13.61	14.13	14.07	1.19	1.05
ERDŐS-RÉNYI	13.89	14.11	14.12	0.76	0.77
BARABÁSI-ALBERT	13.52	14.51	14.59	0.84	0.92
BIPARTITE	9.96	9.98	9.99	0.85	1.08

for at all, due to the uneven degree distribution of the graphs.

4.3.6. Planar Extraction

As already mentioned in Section 4.3.1, the additional removal of extra edges in the form of PLEX (as proposed in Section 2.7) has been shown to save space on 10/11 tested graphs. The only time we did not see any improvements was on the bipartite graph, where the maximum planar subgraph approximation (extracting a triangulated cactus) could only extract a basic spanning tree, due to the requirement of triangles. This also already showcased a worst-case scenario for PLEX compared to TREX, but because the cactus representation requires barely more than 2 bits per edge in the worst case, this only led to a penalty of 0.002%, being non-visible when rounded to two digits.

In our case of planar subgraph extraction, only a cactus graph was removed, which can contain at most $\lfloor \frac{3(n-1)}{2} \rfloor$ edges. This led to, on average, only $1.27n$ extracted edges on our test graphs, which already showed noticeable space savings nonetheless, while still leaving much room for improvement – specifically if one managed to extract a planar subgraph without the constraint of it also being a cactus.

4.3.7. Twin Removal

While the focus of this thesis is edge extraction, additional space savings could potentially be achieved by twin removal [ANNW26]. Removing vertices that share an identical neighborhood and only representing them with a single representative node can, for example, drastically reduce entropy on specific biological graphs, created using a duplication model [VFMV01, ANNW26]. To test whether this could be of any use on

arbitrary real-world graphs, the reduction from n to the number of representative twin classes n_0 was calculated on all test networks. As expected, n_0 heavily depends on the graph’s domain and topology. Most real-world networks did not show any mentionable redundancy, however, WIKI-VOTE contained 5847 twin classes, being nearly 18% less than n . This also appears intuitive, because in votings, each person is given only a few decisions, where groups with similar interests suggest the tendency to cast votes for the same candidates as well.

On all the other seven real-world graphs we saw an averaged reduction of just 1.64%, a comparatively modest redundancy. The corresponding overhead of $\log_2 \binom{n}{n_0}$ to store the size of each twin class was calculated for each graph and has been shown to be small throughout, averaging just 0.17% of the total TREX-compressed size across all 11 networks.

However, the vertex label reduction from n to n_0 alone does not determine the actual space reduction of the residual graph, so whether the corresponding savings of D , T , and mainly the residual entropy make up for this overhead remains unmeasured.

Having a short look at the synthetic graphs specifically, we notice that for both ERDŐS-RÉNYI and BARABÁSI-ALBERT, not a single non-trivial twin class was found. However, extreme twin redundancy was visible on BIPARTITE, which, by definition, consists of only two twin classes. As we have seen little to no improvement from either TREX or PLEX on this graph, twin removal could compress it to virtually zero space per edge.

While the actual savings for each of those graphs and the harmony with TREX remain to be measured, some graphs suggest that the twin reduction step prior to TREX could offer additional room for compression, specifically in combination with the small overhead it costs.

5. Conclusion

5.1. Summary

This thesis provided a comprehensive empirical evaluation of Tree Extraction (TREX) for lossless graph compression. Through a proof-of-concept implementation, which isolates compression potential from low-level implementation, TREX was evaluated across different real-world and synthetic benchmark networks.

We saw that TREX’s navigational logic generally worked well, with only a few flaws being identified and corrected in the process. Additionally, all the operations – if succinct underlying structures were used – would execute at their expected time complexity, since they preserve the same query sequence of the underlying structures. Also, TREX consistently reduced the space required to store a graph across all test datasets over the benchmark wavelet tree representation [Nav14], confirming its viability as a compression method. The evaluation of edge orientation strategies for undirected graphs and different approaches of choosing a spanning tree showed that the respective greedy algorithms consistently provided the best results. Still, the gap between those strategies was shown to be comparatively small, indicating that the primary TREX compression comes from the raw extraction of edges itself.

To predict this compression performance before TREX construction, graph density has proven to be a highly accurate predictor, outperforming the mathematically proven bounds DUB and NIB on $H(G - T)$ [ANNW26], if used as predictors. The bounds are conservative by construction and consistently overestimated the residual entropy by roughly $n \log_2 n$.

Furthermore, the twin removal introduced by Alaoui et al. did not offer consistent benefits, but could potentially be worth including as a preprocessing step, due to the small overhead it costs.

Finally, the potential of PLEX was tested, a logical continuation of TREX, where one tries to extract more than just a regular spanning tree. It showed very consistent improvements throughout but does not quite live up to its potential, due to the used planar approximation only being able to extract at most $\sim 1.5n$ edges yet. Ultimately, those results confirm that TREX yields consistent space compression throughout, which can in practice be reliably estimated ahead of any computation using basic graph metadata.

5.2. Future Work

From the results of this thesis, multiple directions for future work follow naturally.

The most immediate step is the modification of the developed proof-of-concept implementation to make it fully succinct. By replacing the three underlying structures with true succinct implementations and removing some evaluation overhead in the current implementation, one could validate the space estimates reported in this thesis against actual memory consumption. Also, then the exact query time of TREX's $\mathcal{O}(\log n)$ navigation could be better quantified and measured on real-world datasets.

A second direction certainly is regarding the planar subgraph extraction. As discussed in the evaluation, the fraction of extracted edges has the most meaningful impact on the compression of real-world graphs. Two paths seem promising here: The current cactus-based approach could be worked out more thoroughly — in particular, coming up with a good solution for the canonical relabeling scheme for the representation by Chakraborty et al. [CJSS24] and crafting a way to efficiently navigate the resulting structure — after which it could be prototyped and benchmarked, as it was done for TREX in this thesis. Alternatively, one could pursue the search for a different structurally constrained subgraph, such as a series-parallel graph (at most $2n - 3$ edges) or utilize a closer approximation to the maximum planar subgraph problem. However, a new succinct representation and labeling scheme for the new subgraph would need to be thought of as well.

And while the density-based regression model was a good predictor of TREX compressibility on the test datasets, it lacks generalizability in edge cases. Future work could expand the sample size to experiment with more complex models or simply try a multiple variable regression. By introducing additional regressors — while preserving the simplicity of the model — edge cases could be recognized and properly approximated.

References

- [ANNW26] Ziad Ismaili Alaoui, Tamio-Vesa Nakajima, Namrata, and Sebastian Wild. Rooting out entropy: Optimal tree extraction for ultra-succinct graphs, 2026. URL: <https://arxiv.org/abs/2603.14649>, arXiv:2603.14649.
- [BA99] Albert-László Barabási and Réka Albert. Emergence of scaling in random networks. *Science*, 286(5439):509–512, October 1999. doi:10.1126/science.286.5439.509.
- [BGP⁺19] Digvijay Boob, Yu Gao, Richard Peng, Saurabh Sawlani, Charalampos E. Tsourakakis, Di Wang, and Junxing Wang. Flowless: Extracting densest subgraphs without flow computations, 2019. URL: <https://arxiv.org/abs/1910.07087>, arXiv:1910.07087.
- [CFFK98] Gruia Cuaresma, Cristina G. Fernandes, Ulrich Finkler, and Howard J. Karloff. A better approximation algorithm for finding planar subgraphs. *J. Algorithms*, 27(2):269–302, 1998. doi:10.1006/JAGM.1997.0920.
- [CFJ08] Jean Cardinal, Samuel Fiorini, and Gwenaél Joret. Minimum entropy orientations. *Operations Research Letters*, 36(6):680–683, November 2008. doi:10.1016/j.orl.2008.06.010.
- [Cha00] Moses Charikar. Greedy approximation algorithms for finding dense components in a graph. In *International Workshop on Approximation Algorithms for Combinatorial Optimization*, pages 84–95. Springer, 2000. URL: <https://users.ics.aalto.fi/gionis/greedy-dense.pdf>.
- [CJSS24] Sankardeep Chakraborty, Seungbum Jo, Kunihiro Sadakane, and Srinivasa Rao Satti. Succinct data structures for sp, block-cactus and 3-leaf power graphs. *Int. J. Found. Comput. Sci.*, 35(6):705–722, 2024. doi:10.1142/S012905412341006X.
- [CKL⁺22] Li Chen, Rasmus Kyng, Yang P. Liu, Richard Peng, Maximilian Probst Gutenberg, and Sushant Sachdeva. Maximum flow and minimum-cost flow in almost-linear time, 2022. URL: <https://arxiv.org/abs/2203.00671>, arXiv:2203.00671.
- [CLM19] Cristian Consonni, David Laniado, and Alberto Montresor. Wikilinkgraphs: A complete, longitudinal and multilanguage dataset of the wikipedia link networks, January 2019. doi:10.5281/zenodo.2539424.
- [CQT22] Chandra Chekuri, Kent Quanrud, and Manuel R. Torres. Densest subgraph: Supermodularity, iterative peeling, and flow. In Joseph (Seffi) Naor and Niv Buchbinder, editors, *Proceedings of the 2022 ACM-SIAM Symposium on Discrete Algorithms, SODA 2022, Virtual Conference / Alexandria, VA, USA, January 9 - 12, 2022*, pages 1531–1555. SIAM, 2022. doi:10.1137/1.9781611977073.64.

- [CU24] Gruia Călinescu and Sumedha Uniyal. Local Optimization Algorithms for Maximum Planar Subgraph. In Timothy Chan, Johannes Fischer, John Iacono, and Grzegorz Herman, editors, *32nd Annual European Symposium on Algorithms (ESA 2024)*, volume 308 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 38:1–38:18, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. doi:10.4230/LIPIcs.ESA.2024.38.
- [Deva] NetworkX Developers. https://networkx.org/documentation/stable/reference/algorithms/generated/networkx.algorithms.bipartite.generators.complete_bipartite_graph.html.
- [Devb] NetworkX Developers. https://networkx.org/documentation/stable/_modules/networkx/algorithms/approximation/density.html.
- [ER59] P Erdős and A Rényi. On random graphs i. *Publicationes Mathematicae Debrecen*, 6:290–297, 1959.
- [FFG⁺20] Leo Ferres, José Fuentes-Sepúlveda, Travis Gagie, Meng He, and Gonzalo Navarro. Fast and compact planar embeddings. *Comput. Geom.*, 89:101630, 2020. doi:10.1016/J.COMGEO.2020.101630.
- [GGK03] Johannes Gehrke, Paul Ginsparg, and Jon M. Kleinberg. Overview of the 2003 KDD cup. *SIGKDD Explor.*, 5(2):149–151, 2003. doi:10.1145/980972.980992.
- [GGV02] Roberto Grossi, Ankur Gupta, and Jeffrey Vitter. High-order entropy-compressed text indexes. *Proceedings of the Annual ACM-SIAM Symposium on Discrete Algorithms*, 11 2002. doi:10.1145/644108.644250.
- [Gil59] E. N. Gilbert. Random graphs. *The Annals of Mathematical Statistics*, 30(4):1141–1144, 1959. doi:10.1214/aoms/1177706098.
- [Gol84] Andrew V. Goldberg. Finding a maximum density subgraph. Technical report, University of California, Berkeley, Berkeley, CA, 1984. URL: <https://digincoll.lib.berkeley.edu/record/136696/files/CSD-84-171.pdf>.
- [Jac89] Guy Jacobson. Space-efficient static trees and graphs. In *30th Annual Symposium on Foundations of Computer Science, Research Triangle Park, North Carolina, USA, 30 October - 1 November 1989*, pages 549–554. IEEE Computer Society, 1989. doi:10.1109/SFCS.1989.63533.
- [LAH07] Jure Leskovec, Lada A. Adamic, and Bernardo A. Huberman. The dynamics of viral marketing. *ACM Trans. Web*, 1(1):5, 2007. doi:10.1145/1232722.1232727.
- [LHK10a] Jure Leskovec, Daniel P. Huttenlocher, and Jon M. Kleinberg. Predicting positive and negative links in online social networks. In Michael Rappa,

- Paul Jones, Juliana Freire, and Soumen Chakrabarti, editors, *Proceedings of the 19th International Conference on World Wide Web, WWW 2010, Raleigh, North Carolina, USA, April 26-30, 2010*, pages 641–650. ACM, 2010. doi:10.1145/1772690.1772756.
- [LHK10b] Jure Leskovec, Daniel P. Huttenlocher, and Jon M. Kleinberg. Signed networks in social media. In Elizabeth D. Mynatt, Don Schoner, Geraldine Fitzpatrick, Scott E. Hudson, W. Keith Edwards, and Tom Rodden, editors, *Proceedings of the 28th International Conference on Human Factors in Computing Systems, CHI 2010, Atlanta, Georgia, USA, April 10-15, 2010*, pages 1361–1370. ACM, 2010. doi:10.1145/1753326.1753532.
- [LK14] Jure Leskovec and Andrej Krevl. SNAP Datasets: Stanford large network dataset collection. <http://snap.stanford.edu/data>, June 2014.
- [LKF05] Jure Leskovec, Jon M. Kleinberg, and Christos Faloutsos. Graphs over time: densification laws, shrinking diameters and possible explanations. In Robert Grossman, Roberto J. Bayardo, and Kristin P. Bennett, editors, *Proceedings of the Eleventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Chicago, Illinois, USA, August 21-24, 2005*, pages 177–187. ACM, 2005. doi:10.1145/1081870.1081893.
- [LLDM09] Jure Leskovec, Kevin J. Lang, Anirban Dasgupta, and Michael W. Mahoney. Community structure in large networks: Natural cluster sizes and the absence of large well-defined clusters. *Internet Math.*, 6(1):29–123, 2009. doi:10.1080/15427951.2009.10129177.
- [Nav14] Gonzalo Navarro. Wavelet trees for all. *Journal of Discrete Algorithms*, 25:2–20, 2014. 23rd Annual Symposium on Combinatorial Pattern Matching. doi:10.1016/j.jda.2013.07.004.
- [Nav16] Gonzalo Navarro. *Compact Data Structures - A Practical Approach*. Cambridge University Press, 2016.
- [RA15] Ryan A. Rossi and Nesreen K. Ahmed. The network data repository with interactive graph analytics and visualization. In *AAAI*, 2015. URL: <https://networkrepository.com>.
- [RRS07] Rajeev Raman, Venkatesh Raman, and Srinivasa Rao Satti. Succinct indexable dictionaries with applications to encoding k -ary trees, prefix sums and multisets. *ACM Transactions on Algorithms*, 3(4):43, November 2007. doi:10.1145/1290672.1290680.
- [RS21] Benedek Rozemberczki and Rik Sarkar. Twitch gamers: a dataset for evaluating proximity preserving and structural role-based node embeddings, 2021. arXiv:2101.03091.

-
- [Tur84] György Turán. On the succinct representation of graphs. *Discret. Appl. Math.*, 8(3):289–294, 1984. doi:10.1016/0166-218X(84)90126-4.
- [VFMV01] A. Vazquez, A. Flammini, A. Maritan, and A. Vespignani. Modeling of protein interaction networks, 2001. URL: <https://arxiv.org/abs/cond-mat/0108043>, arXiv:cond-mat/0108043.
- [Wes01] Douglas B. West. *Introduction to Graph theory*. Pearson Education, 2 edition, 2001.

A. Directed Queries

This appendix contains the complete, tested, and zero-based Python implementation of the directed TREX navigational operations. Each method assumes an already constructed TREX instance, consisting of the LOUDS tree T , direction bitvector D , and residual structures in the form of a wavelet tree A' and boundary bitvector S' . The construction logic implemented by a builder, as described in Section 3.1, is omitted here and provided in full in the accompanying GitHub repository.

```

1  from src.dummy.dummy_louds import DummyLOUDs
2  from src.dummy.dummy_wavelet import DummyWaveletTree
3  from src.dummy.dummy_bitvector import DummyBitvector
4
5
6  class DirectedTrexGraph:
7
8      def __init__(self, T: DummyLOUDs, A_prime: DummyWaveletTree, S_prime:
9          ↳ DummyBitvector, D: DummyBitvector):
10         self.T = T
11         self.A_prime = A_prime
12         self.S_prime = S_prime
13         self.D = D
14
15     def outdegree(self, v: int) -> int:
16         # how many zeros (out-edges) there are after our 1 to the next 1
17         outgoing_A_prime = self.S_prime.select(v + 1, 1) -
18         ↳ self.S_prime.select(v, 1) - 1
19         outgoing_T = 0
20         # we check if there's an edge pointing upwards
21         if self.D.access(v - 1) == 0 and self.T.parent(v) != 0:
22             outgoing_T += 1
23
24         # we check how many edges point downwards to the children, meaning we
25         ↳ count the ones in the interval of children within D
26         if self.T.degree(v) != 0:
27             c_1 = self.T.child(v, 1)
28             c_k = self.T.child(v, self.T.degree(v))
29             outgoing_T += self.D.rank(c_k - 1, 1) - self.D.rank(c_1 - 2, 1)
30         return outgoing_A_prime + outgoing_T
31
32     def indegree(self, v: int) -> int:
33         ingoing_A_prime = self.A_prime.rank(len(self.A_prime) - 1, v)

```

```

31
32     ingoing_T = 0
33
34     if self.D.access(v-1) == 1 and self.T.parent(v) != 0:
35         ingoing_T += 1
36
37     if self.T.degree(v) != 0:
38         c_1 = self.T.child(v,1)
39         c_k = self.T.child(v, self.T.degree(v))
40         ingoing_T += self.D.rank(c_k - 1, 0) - self.D.rank(c_1 - 2, 0)
41
42     return ingoing_A_prime + ingoing_T
43
44 def adjacent(self, u: int, v: int) -> bool:
45     if self.T.parent(u) == v or self.T.parent(v) == u:
46         return True
47
48     # we simply check if the rank(v)/rank(u) changes within the
49     ↪ outneighborhood of u/v within A_prime.
50     # also small typo in the paper, should be v/u rather than 1, also
51     ↪ outdegree(u) is the wrong metric, need to use outdegree -
52     ↪ T_outdegree
53     A_prime_outdegree_u = self.S_prime.select(u + 1, 1) -
54     ↪ self.S_prime.select(u, 1) - 1
55     s = self.S_prime.select(u,1) - u
56     if self.A_prime.rank(s + A_prime_outdegree_u, v) - self.A_prime.rank(s,
57     ↪ v) >= 1:
58         return True
59
60     A_prime_outdegree_v = self.S_prime.select(v + 1, 1) -
61     ↪ self.S_prime.select(v, 1) - 1
62     s_dash = self.S_prime.select(v,1) - v
63     if self.A_prime.rank(s_dash + A_prime_outdegree_v, u) -
64     ↪ self.A_prime.rank(s_dash, u) >= 1:
65         return True
66     return False
67
68 def outneighbor (self, v: int, i: int) -> int:
69     if i > self.outdegree(v) or i < 1:
70         return -1
71
72

```

```

65     # we calculate the outdegree of v in T by first checking the node to
66     ↪ the parent and then counting the number of children with a 1 in D
67     T_outdegree = 0
68     if self.D.access(v-1) == 0 and self.T.parent(v) != 0:
69         T_outdegree += 1
70     if self.T.degree(v) > 0:
71         child_last = self.T.child(v, self.T.degree(v))
72         child_first = self.T.child(v, 1)
73         T_outdegree += self.D.rank(child_last - 1, 1) -
74         ↪ self.D.rank(child_first - 2, 1)
75
76     j = i
77     # return parent in T if asked for
78     if i == 1 and self.D.access(v-1) == 0 and self.T.parent(v) != 0:
79         return self.T.parent(v)
80     # otherwise pick i-th or (i-1)-th outgoing child in T
81     if i <= T_outdegree and self.T.degree(v) != 0:
82         if self.D.access(v-1) == 0 and self.T.parent(v) != 0:
83             j = i-1
84             # first child
85             c_1 = self.T.child(v, 1)
86             # number of ones that appear exclusively before the first child in
87             ↪ d
88             o = self.D.rank(c_1 - 2, 1)
89             # we select the correct one in D to get the position of the i'th
90             ↪ outchild and add 1 to make it a proper label again
91             return self.D.select(j + o, 1) + 1
92
93     # if it's not in T, we look for it in A_prime by simply calculating the
94     ↪ outneighbor index it must be in A_prime
95     # then by using S_prime we get the number of 0's before our edge,
96     ↪ telling us where the outneighborhood begins in A_prime
97     j = i - T_outdegree
98     s = self.S_prime.select(v, 1) - v
99     return self.A_prime.access(s + j)
100
101 def inneighbor(self, v:int, i: int) -> int:
102     if i > self.indegree(v) or i < 1:
103         return -1
104
105     has_ingoiing_parent = (self.D.access(v-1) == 1 and self.T.parent(v) !=
106     ↪ 0)

```

```

100
101     T_indegree = int(has_ingoiing_parent)
102
103     if self.T.degree(v) > 0:
104         child_last = self.T.child(v, self.T.degree(v))
105         child_first = self.T.child(v, 1)
106         T_indegree += self.D.rank(child_last - 1, 0) -
            ↪ self.D.rank(child_first - 2, 0)
107     j = i
108     if i == 1 and has_ingoiing_parent:
109         return self.T.parent(v)
110     if i <= T_indegree and self.T.degree(v) != 0:
111         if has_ingoiing_parent:
112             j = i-1
113             c_1 = self.T.child(v, 1)
114             o = self.D.rank(c_1 - 2, 0)
115             return self.D.select(j + o, 0) + 1
116
117     # the rest was parallel to outneighbor, but here we now have to again
118     ↪ calculate j, the index of the inneighbor we are looking for in
119     ↪ A_prime. Then we
120     # select the position of that edge in A_prime, then the corresponding
121     ↪ position in S_prime, and lastly we only need to calculate what node
122     ↪ this belongs to in S_prime.
123
124     j = i - T_indegree
125     y = self.A_prime.select(j, v)
126     a = self.S_prime.select(y + 1, 0)
127     return self.S_prime.rank(a, 1)
128
129 def outneighbor_rank(self, v: int, w: int) -> int:
130
131     # directed adjacency check
132     if self.is_edge(v, w) == False:
133         return -1
134
135     if self.T.parent(v) == w and self.D.access(v-1) == 0 and
        ↪ self.T.parent(v) != 0:
136         return 1
137
138     # if w is one of v's children
139     if self.T.parent(w) == v and self.D.access(w-1) == 1:

```

```

136         c_1 = self.T.child(v,1)
137         j = self.D.rank(w - 1,1) - self.D.rank(c_1 - 2, 1)
138         if self.D.access(v-1) == 0 and self.T.parent(v) != 0:
139             j += 1
140         return j
141
142     T_outdegree = 0
143     if self.D.access(v-1) == 0 and self.T.parent(v) != 0:
144         T_outdegree += 1
145     if self.T.degree(v) > 0:
146         child_last = self.T.child(v, self.T.degree(v))
147         child_first = self.T.child(v,1)
148         T_outdegree += self.D.rank(child_last - 1, 1) -
            ↪ self.D.rank(child_first - 2 ,1)
149
150     # s marks the start of v's outneighborhood in A_prime
151     s = self.S_prime.select(v,1) - v
152     # count how many times w already occurs in A_prime before v's
153     ↪ outneighborhood begins
154     r = self.A_prime.rank(s, w)
155     # then we find the next occurrence of w and shift it by s again to get
156     ↪ w's rank among v's outneighbors
157     j = self.A_prime.select(r + 1 , w) - s
158     return j + T_outdegree
159
160 def inneighbor_rank(self, v: int, w: int) -> int:
161
162     if self.is_edge(w, v) == False:
163         return -1
164
165     if self.T.parent(v) == w and self.D.access(v-1) == 1 and
166     ↪ self.T.parent(v) != 0:
167         return 1
168
169     if self.T.parent(w) == v and self.D.access(w-1) == 0:
170         c_1 = self.T.child(v,1)
171         j = self.D.rank(w - 1,0) - self.D.rank(c_1 - 2, 0)
172         if self.D.access(v-1) == 1 and self.T.parent(v) != 0:
173             j += 1
174         return j
175
176     T_indegree = 0

```

```

174         if self.D.access(v-1) == 1 and self.T.parent(v) != 0:
175             T_indegree += 1
176         if self.T.degree(v) > 0:
177             child_last = self.T.child(v, self.T.degree(v))
178             child_first = self.T.child(v, 1)
179             T_indegree += self.D.rank(child_last - 1, 0) -
                ↪ self.D.rank(child_first - 2, 0)
180
181         # calculate where the outneighborhood of w starts within A_prime
182         s = self.S_prime.select(w, 1) - w
183         # j are the occurrences of w in A_prime up until (inclusive) (v -> w)
184         j = self.A_prime.rank(s, v) + 1
185
186         return j + T_indegree
187
188         # small helper function to determine whether a directed edge exists,
189         ↪ not explicitly described in the paper
189     def is_edge(self, u: int, v: int) -> bool:
190
191         # check in the Tree
192         if self.T.parent(u) == v and self.D.access(u - 1) == 0:
193             return True
194         if self.T.parent(v) == u and self.D.access(v - 1) == 1:
195             return True
196
197         # check in A_prime
198         s = self.S_prime.select(u, 1) - u
199         A_prime_outdegree_u = self.S_prime.select(u + 1, 1) -
            ↪ self.S_prime.select(u, 1) - 1
200
201         if self.A_prime.rank(s + A_prime_outdegree_u, v) - self.A_prime.rank(s,
            ↪ v) >= 1:
202             return True
203         return False

```

B. Undirected Queries

Analogous to Appendix A, this appendix contains the full implementation of the added queries for undirected TREX instances, which no longer store the direction bitvector D .

```

1  from src.dummy.dummy_louds import DummyLouds
2  from src.dummy.dummy_wavelet import DummyWaveletTree
3  from src.dummy.dummy_bitvector import DummyBitvector
4
5
6  class UndirectedTrexGraph:
7
8      def __init__(self, T: DummyLouds, A_prime: DummyWaveletTree, S_prime:
9          ↪ DummyBitvector):
10         self.T = T
11         self.A_prime = A_prime
12         self.S_prime = S_prime
13
14     def degree(self, v: int) -> int:
15         # number of children within the tree
16         T_degree = self.T.degree(v)
17         # one must be added if v is not a root
18         if self.T.parent(v) != 0:
19             T_degree += 1
20         s = self.S_prime.select(v, 1)
21         s_dash = self.S_prime.select(v + 1, 1)
22         A_prime_outdegree = s_dash - s - 1
23         A_prime_indegree = self.A_prime.rank(len(self.A_prime) - 1, v)
24         return T_degree + A_prime_outdegree + A_prime_indegree
25
26     def adjacent(self, u: int, v: int) -> bool:
27         if self.T.parent(u) == v or self.T.parent(v) == u:
28             return True
29
30         # we simply check if the rank(v/u) changes within the outneighborhood
31         ↪ of u/v in A_prime.
32         A_prime_outdegree_u = self.S_prime.select(u + 1, 1) -
33             ↪ self.S_prime.select(u, 1) - 1
34         s = self.S_prime.select(u, 1) - u
35         if self.A_prime.rank(s + A_prime_outdegree_u, v) - self.A_prime.rank(s,
36             ↪ v) >= 1:
37             return True

```

```

34     A_prime_outdegree_v = self.S_prime.select(v + 1, 1) -
    ↪ self.S_prime.select(v, 1) - 1
35     s_dash = self.S_prime.select(v, 1) - v
36     if self.A_prime.rank(s_dash + A_prime_outdegree_v, u) -
    ↪ self.A_prime.rank(s_dash, u) >= 1:
37         return True
38     return False
39
40     # first tree neighbors, then A_prime "outneighbors", then A_prime
    ↪ "inneighbors"
41     def neighbor(self, v: int, i: int) -> int:
42         j = i
43         T_degree = self.T.degree(v)
44
45         # check whether we are simply asking for the parent
46         if i == 1 and self.T.parent(v) != 0:
47             return self.T.parent(v)
48
49         # increase the degree if v has a parent within the tree, decrease the
    ↪ neighbor we are looking for by one
50         if self.T.parent(v) != 0:
51             j -= 1
52             T_degree += 1
53         # within the Tree neighbors
54         if i <= T_degree:
55             return self.T.child(v, j)
56
57         # checking "outneighbors" in A_prime
58         remaining = i - T_degree
59         s = self.S_prime.select(v, 1)
60         s_dash = self.S_prime.select(v+1, 1)
61         A_prime_outdeg = s_dash - s - 1
62
63         if remaining <= A_prime_outdeg:
64             A_prime_pos = s - v + 1
65             return self.A_prime.access(A_prime_pos + remaining - 1)
66
67         # checking "inneighbors" in A_prime
68         remaining -= A_prime_outdeg
69         if remaining <= self.A_prime.rank(len(self.A_prime) - 1, v):
70             # position in A_prime where the desired edge lies
71             pos_A = self.A_prime.select(remaining, v)

```

```

72         # position in S_prime where this edge is declared
73         pos_S = self.S_prime.select(pos_A+1, 0)
74         # finally the vertex, to which this edge belongs
75         return self.S_prime.rank(pos_S, 1)
76
77     return -1
78
79     def neighbor_rank(self, v: int, w: int) -> int:
80
81         if self.adjacent(v, w) == False:
82             return -1
83
84         # checking if w is v's parent and adjusting T_degree
85         T_degree = self.T.degree(v)
86         if self.T.parent(v) != 0:
87             T_degree += 1
88             if self.T.parent(v) == w:
89                 return 1
90
91         # checking if w is one of v's children
92         if self.T.parent(w) == v:
93             T_rank = self.T.child_rank(w)
94             if self.T.parent(v) != 0:
95                 T_rank += 1
96             return T_rank
97
98         s = self.S_prime.select(v, 1)
99         s_dash = self.S_prime.select(v + 1, 1)
100         A_prime_outdegree = s_dash - s - 1
101         A_prime_pos = s - v + 1
102
103         # check if w is an outneighbor in A_prime
104         if self.A_prime.rank(A_prime_pos - 1, w) !=
105             ↪ self.A_prime.rank(A_prime_pos + A_prime_outdegree - 1, w):
106             # the position of the edge v -> w in A_prime
107             pos = self.A_prime.select(self.A_prime.rank(A_prime_pos - 1, w) + 1,
108                 ↪ w)
109             difference = pos - A_prime_pos + 1
110             return difference + T_degree
111
112         # now w must be within the inneighbors of v
113         pos_A_prime = self.S_prime.select(w + 1, 1) - w

```

```
112         num_of_occ = self.A_prime.rank(pos_A_prime - 1, v)
113         return T_degree + A_prime_outdegree + num_of_occ
114
```

Erklärung

Hiermit erkläre ich, dass ich meine Bachelorarbeit mit dem Thema "Empirical Evaluation of Tree Extraction" selbstständig verfasst sowie alle wesentlichen Quellen und Hilfsmittel angegeben habe.

Ort, Datum

Julius Reitlinger (Unterschrift)